BTGenerator

Décisions d'architecture

Document de référence pour les choix techniques
du pipeline de fine-tuning de Behavior Trees
pour la stack NAV4RAIL

Équipe BTGenerator — SNCF

4 mars 2026

Ce document couvre cinq axes de décision : jeu de données synthétique (proxy), rejet du RAG, sécurité et watchdog, fine-tuning LoRA, et évaluation des Behavior Trees.

Table des matières

1 Introduction **3**
1.1 Glossaire des termes et acronymes . . . 3

2 Axe 1 — Jeu de données synthétique (proxy) **5**
2.1 Contexte . . . 5
2.2 Décision . . . 5
2.3 Méthode de génération . . . 5
2.3.1 Décomposition en blocs fonctionnels . . . 5
2.3.2 Composition en templates . . . 5
2.3.3 Randomisation des missions . . . 6
2.4 Évolution du jeu de données . . . 6
2.5 Les 8 catégories du dataset v4 scaled (2 000 exemples) . . . 7
2.6 Justification et limites . . . 7

3 Axe 2 — RAG : analyse et rejet motivé **8**
3.1 Contexte . . . 8
3.2 Décision . . . 8
3.3 Analyse quantitative . . . 8
3.4 Alternatives écartées . . . 8
3.5 Architecture retenue . . . 9

4 Axe 3 — Sécurité et watchdog : défense en profondeur **10**
4.1 Contexte . . . 10
4.2 Décision . . . 10
4.3 Vue d'ensemble du pipeline . . . 10
4.4 Couche 1 : Décodage contraint (niveau token) . . . 10
4.5 Couche 2 : Extraction XML (post-traitement) . . . 11
4.6 Couche 3 : Validateur à trois niveaux . . . 11

5 Axe 4 — Fine-tuning LoRA : choix techniques **12**
5.1 Contexte . . . 12
5.2 Décision : QLoRA 4-bit NF4 . . . 12
5.3 Paramètres d'entraînement . . . 12
5.4 Importance du masquage de la perte . . . 12
5.5 Choix du modèle . . . 12
5.6 Résultats . . . 13

6 Axe 5 — Évaluation : syntaxe, structure, sémantique **14**
6.1 Contexte . . . 14
6.2 Décision . . . 14
6.3 Niveau L1 — Syntaxique (bloquant) . . . 14
6.4 Niveau L2 — Structurel (bloquant) . . . 15

6.5 Niveau L3 — Sémantique (avertissements) 15
6.6 Protocole de test 15

7 Tableau récapitulatif **17**

8 Catalogue des 27 skills NAV4RAIL **17**

1 Introduction

Le projet **BTGenerator** vise à permettre à un opérateur de décrire une mission ferroviaire en langage naturel, puis de générer automatiquement un **Behavior Tree** (arbre de comportement, ou **BT**) au format XML compatible avec la bibliothèque **BehaviorTree.CPP v4**. Ces BT sont destinés à la stack **NAV4RAIL** — l'équivalent ferroviaire de Nav2 (ROS 2) développé par la SNCF pour piloter ses robots de maintenance sur le réseau ferré.

Le pipeline repose sur le **fine-tuning** (ajustement fin) d'un modèle de langage (**LLM**, pour *Large Language Model* — grand modèle de langage) avec la technique **QLoRA** (une méthode pour entraîner seulement une petite fraction des paramètres du modèle, tout en gardant le reste gelé et compressé en 4 bits).

Ce document consolide les cinq décisions architecturales majeures du projet :

1. **Axe 1 — Jeu de données synthétique (proxy)** : comment 2 000 exemples d'entraînement ont été générés à partir d'un seul exemple XML réel.
2. **Axe 2 — RAG : analyse et rejet motivé** : pourquoi la recherche augmentée par récupération de documents (*RAG*) a été écartée.
3. **Axe 3 — Sécurité et watchdog** : l'architecture de défense en profondeur à trois couches.
4. **Axe 4 — Fine-tuning LoRA** : les choix techniques d'entraînement.
5. **Axe 5 — Évaluation** : le validateur à trois niveaux (syntaxe, structure, sémantique).

1.1 Glossaire des termes et acronymes

BT — **Behavior Tree** — arbre de comportement. Structure arborescente qui décompose une tâche complexe en sous-tâches (nœuds). Utilisé en robotique et dans les jeux vidéo.

BehaviorTree.CPP — Bibliothèque C++ de référence pour exécuter des BT. Le format v4 est le standard XML utilisé par la stack NAV4RAIL.

NAV4RAIL — Stack logicielle de navigation ferroviaire développée par la SNCF, analogue à Nav2 (ROS 2) mais adaptée aux robots de maintenance sur rail. BTGenerator produit des BT au format attendu par NAV4RAIL.

LLM — **Large Language Model** — grand modèle de langage. Réseau de neurones pré-entraîné sur de vastes corpus de texte, capable de générer du texte (ici, du XML).

Fine-tuning — Ajustement fin : on reprend un LLM pré-entraîné et on l'entraîne sur un petit jeu de données spécialisé pour qu'il apprenne une tâche précise.

LoRA — **Low-Rank Adaptation** — adaptation de rang faible. Technique qui ajoute de petites matrices entraînables à côté des poids gelés du modèle, réduisant considérablement la mémoire requise.

QLoRA — **Quantized LoRA** — LoRA quantifié. Variante de LoRA où les poids du modèle de base sont compressés en 4 bits (*NF4*, un format de quantification

optimisé pour les distributions normales des poids). Permet d'entraîner un modèle de 7 milliards de paramètres sur un GPU de 16 Go.

NF4 **NormalFloat 4-bit** — format de quantification 4 bits conçu pour les poids de réseaux de neurones, dont la distribution suit approximativement une loi normale.

RAG **Retrieval-Augmented Generation** — génération augmentée par la récupération. Architecture où le modèle consulte une base de connaissances externe avant de générer sa réponse.

GBNF **GGML BNF** — format de grammaire (dérivé de la forme Backus-Naur) utilisé pour contraindre la génération de texte token par token. Chaque token généré doit respecter la grammaire.

SLURM Gestionnaire de tâches pour clusters de calcul.

VRAM Mémoire vidéo du GPU. Facteur limitant principal pour l'entraînement de LLM.

Proxy (jeu de données)
Ici, *proxy* désigne le jeu de données synthétique qui *remplace* (sert de substitut à) des données réelles annotées à la main.

Skill Compétence atomique du robot (ex. : `<LoadMission/>`, `<Move/>`). Chaque skill est un nœud feuille dans le BT.

Sequence Nœud de contrôle du BT : exécute ses enfants de gauche à droite, s'arrête au premier échec.

Fallback Nœud de contrôle du BT : essaie ses enfants de gauche à droite, s'arrête au premier succès. Utilisé pour les boucles et la gestion d'erreurs.

2 Axe 1 — Jeu de données synthétique (proxy)

2.1 Contexte

Le fine-tuning d'un LLM nécessite des milliers de paires *(mission, BT attendu)*. Or, dans le domaine ferroviaire, il n'existe pas de corpus annoté de Behavior Trees. Annoter manuellement 2 000 exemples serait extrêmement coûteux et sujet à erreurs humaines.

La question centrale était : **comment obtenir suffisamment de données d'entraînement à partir d'un seul exemple XML réel fourni par l'équipe SNCF ?**

2.2 Décision

Générer un **jeu de données synthétique** (le *proxy*) par **composition de patrons** (templates). Un seul exemple réel de Behavior Tree a été décomposé en blocs fonctionnels réutilisables, puis recombiné de multiples façons pour produire 2 000 paires d'entraînement.

2.3 Méthode de génération

La génération repose sur trois piliers :

2.3.1 Décomposition en blocs fonctionnels

L'exemple XML réel a été analysé et décomposé en **blocs réutilisables** :

- **Blocs de préparation** : chargement de mission, validation de structure, calcul d'itinéraire (`<LoadMission/>` → `<MissionStructureValid/>` → `<ProjectPointOnNetwork/>` → `<CreatePath/>` → `<AgregatePath/>`).
- **Boucles de mouvement** : schéma `Fallback(MissionTerminated | Sequence(étape))` — le robot répète une étape tant que la mission n'est pas terminée.
- **Blocs d'inspection** : mesure, analyse, contrôle qualité avec branchement correctif si la qualité est insuffisante.
- **Blocs correctifs** : détection de défauts → génération de sous-séquence corrective → insertion.

Ces blocs sont implémentés sous forme de fonctions Python dans le script `generate_dataset_v4.py`, chacune retournant un fragment XML valide.

2.3.2 Composition en templates

Les blocs sont assemblés en **templates complets** (environ 50 fonctions génératrices) : chaque template représente un type de mission complet. Par exemple :

- `xml_nav_simple()` : préparation + boucle de mouvement + arrêt final.
- `xml_inspect_corrective()` : préparation + boucle d'inspection avec contrôle qualité et séquence corrective.
- `xml_simulation_nav()` : vérification de simulation + navigation complète.

— `xml_complex_nav_measure_return()` : navigation multi-segments + mesures + retour au dépôt.

2.3.3 Randomisation des missions

Chaque exemple est unique grâce à la **variation linguistique** des descriptions de mission. Le générateur combine aléatoirement :

— Des **verbes** : « Déplace-toi », « Va », « Rejoins », « Navigue jusqu'à »
— Des **cibles** : « le dépôt principal », « le poste de maintenance », « la zone de stationnement »
— Des **objets d'inspection** : « la voie », « les rails », « les aiguillages »
— Des **localisations** : « entre le km 15 et le km 28 », « section A », « tunnel nord »

2.4 Évolution du jeu de données

Le dataset a évolué en quatre versions :

Table 1 – Évolution du jeu de données proxy

Version	Exemples	Skills	Templates	Apport principal
v1	100	8 proxy	∼10	Premier prototype, indentation non uniforme
v2	500	8 proxy	∼10	5 catégories, indentation 2 espaces uniforme, XML builder
v3	550	8 proxy	∼10	+50 exemples « inspection sécurisée » (Fallback + CheckObstacle)
v4	550	**27 réels**	∼28	Remplacement des skills proxy par les 27 skills réels NAV4RAIL
v4 scaled	**2 000**	**27 réels**	**∼50**	8 catégories, ∼50 templates, patterns avancés

Le passage v3 → v4 est la transition clé : les 8 skills « proxy » simplifiés (ex. : `GetMission`, `CalculatePath`) ont été remplacés par les 27 skills réels de la stack NAV4RAIL (ex. : `<LoadMission/>`, `<MissionStructureValid/>`, `<ProjectPointOnNetwork/>`, etc.).

La correspondance entre skills proxy et skills réels est documentée dans `SKILLS_CATALOG.md`. Par exemple :

Table 2 – Correspondance skills proxy → skills réels (extrait)

Skill proxy	Skills réels correspondants
`GetMission`	`<LoadMission/>` + `<MissionStructureValid/>`
`CalculatePath`	`<ProjectPointOnNetwork/>` + `<CreatePath/>` + `<AgregatePath/>`
`Move`	`<PassMotionParameters/>` + `<Move/>` (ou `<MoveAndStop/>`)
`ManageMeasurement`	`<ManageMeasurements/>` + `<AnalyseMeasurements/>`
`Stop`	`<MissionTerminated/>` (condition) + `<MoveAndStop/>` (action)

Table 3 – Répartition par catégorie dans le dataset v4 scaled

#	Catégorie	Exemples	Particularité
1	Navigation simple	350	Préparation + mouvement + arrêt
2	Navigation avec autorisation	150	+ `<IsRobotPoseProjectionActive/>`
3	Inspection de voie	350	Boucle mesure/analyse/qualité
4	Inspection avec corrective	200	+ séquence corrective si défaut
5	Mesures simples	150	Mouvement + décélération + mesure
6	Navigation sécurisée	150	Décélération + signalisation
7	Missions complexes	400	Combinaisons multi-phases
8	Simulation	250	Préfixe `<SimulationStarted/>`

2.5 Les 8 catégories du dataset v4 scaled (2 000 exemples)

2.6 Justification et limites

Pourquoi cette approche fonctionne :

— Les BT ont une structure **compositionnelle** : un nombre fini de blocs se combinent pour couvrir tous les cas d'usage.

— La randomisation linguistique assure que le modèle apprend à *comprendre l'intention* de la mission, pas à mémoriser des formulations précises.

— Les 27 skills sont **sans paramètre** (hormis l'attribut `name`) : le modèle doit seulement choisir *quels* skills utiliser et dans *quel ordre*, pas deviner des valeurs numériques.

Quand reconsidérer :

— Si les skills deviennent **paramétrés** (ex. : `sensor="lidar_3d"`, `target_km="8.5"`), la génération synthétique ne suffira plus — il faudra des exemples réels annotés ou un mécanisme RAG.

— Si le nombre de skills dépasse ∼50, la combinatoire des templates deviendra difficile à maintenir manuellement.

Références : `generate_dataset_v4.py`, `SKILLS_CATALOG.md`.

3 Axe 2 — RAG : analyse et rejet motivé

3.1 Contexte

Le **RAG** (*Retrieval-Augmented Generation*, génération augmentée par la récupération) est une architecture populaire où, avant de générer une réponse, le modèle consulte une base de connaissances externe pour récupérer les informations pertinentes. La question était : **faut-il récupérer dynamiquement les skills pertinents à chaque mission, plutôt que de tous les injecter dans le prompt ?**

3.2 Décision

Le RAG a été **rejeté**. À la place, tous les 27 skills sont injectés *statiquement* dans le prompt système, et le décodage contraint (GBNF) empêche structurellement toute hallucination de skill.

3.3 Analyse quantitative

Quatre critères de décision ont été évalués :

Table 4 – Critères de décision Fine-tuning vs. RAG

Critère	Valeur réelle	Seuil RAG	Décision
Nombre de skills	27	> 50	Fine-tuning
Paramètres par skill	1 (`name`)	> 1	Fine-tuning
Fréquence de mise à jour	Stable	$> 1\times$/mois	Fine-tuning
Budget token (27 skills)	∼270 tok (∼7 %)	> 50 %	Fine-tuning

Le calcul est le suivant : 27 skills × ∼10 tokens par skill ≈ 270 tokens, ce qui occupe seulement **7 %** de la fenêtre de contexte de 4 096 tokens de Mistral-7B. Il reste largement assez de place pour le prompt système (∼200 tokens), la description de mission (∼50 tokens), et le BT généré (∼500 tokens).

3.4 Alternatives écartées

Trois alternatives au RAG ont été examinées et écartées :

Option A — Injection sélective par règles Sélection des skills par mots-clés (*« si la mission contient 'inspecte', injecter les skills d'inspection »*). Écartée car fragile face aux missions ambiguës ou multi-catégories (ex. : « inspecte et reviens au dépôt »).

Option B — Grammaire GBNF étendue Ajouter de nouveaux skills à la grammaire GBNF au fil du temps. Déjà partiellement implémentée, mais la maintenance manuelle de la grammaire deviendrait pénible au-delà de ∼80 skills.

Option C — Récupération d'exemples few-shot Pour chaque mission de test, récupérer les 2–3 exemples d'entraînement les plus proches et les inclure dans le prompt. Écartée car cela augmente la longueur du prompt sans apporter de gain mesurable (le modèle fine-tuné génère déjà des BT corrects sans exemples).

3.5 Architecture retenue

L'architecture adoptée combine deux mécanismes :

1. **Prompt statique** : le catalogue complet des 27 skills est injecté dans le prompt système (∼270 tokens, 7 % du contexte).
2. **Décodage contraint** : la grammaire GBNF agit comme un « RAG au niveau du décodage » — à chaque token généré, seuls les tokens correspondant à des skills valides sont autorisés.

Quand reconsidérer :

— Si le nombre de skills dépasse ∼50 (le budget token dépasserait 500 tokens, soit ∼12 % du contexte — toujours acceptable, mais la tendance est à surveiller).

— Si les skills deviennent fortement paramétrés (paramètres dérivés de la mission, ex. : kilomètres, capteurs, seuils).

— Si la fréquence de mise à jour dépasse 1×/mois (chaque modification nécessiterait un re-fine-tuning).

Références : `RAG_ARCHITECTURE.md` (analyse complète), `SKILLS_CATALOG.md`, `nav4rail_grammar.py`.

4 Axe 3 — Sécurité et watchdog : défense en profondeur

4.1 Contexte

Les Behavior Trees générés par le LLM commandent un **robot réel** sur le réseau ferré. Un skill halluciné, un nœud manquant ou un ordre d'exécution incorrect pourrait entraîner un comportement dangereux. La stratégie de sécurité doit être multi-couche : si une couche laisse passer une erreur, la suivante doit la détecter.

4.2 Décision

Architecture de **défense en profondeur à trois couches** :

1. **Couche 1** — Décodage contraint (niveau token)
2. **Couche 2** — Extraction XML (post-traitement)
3. **Couche 3** — Validateur à trois niveaux (vérification complète)

4.3 Vue d'ensemble du pipeline

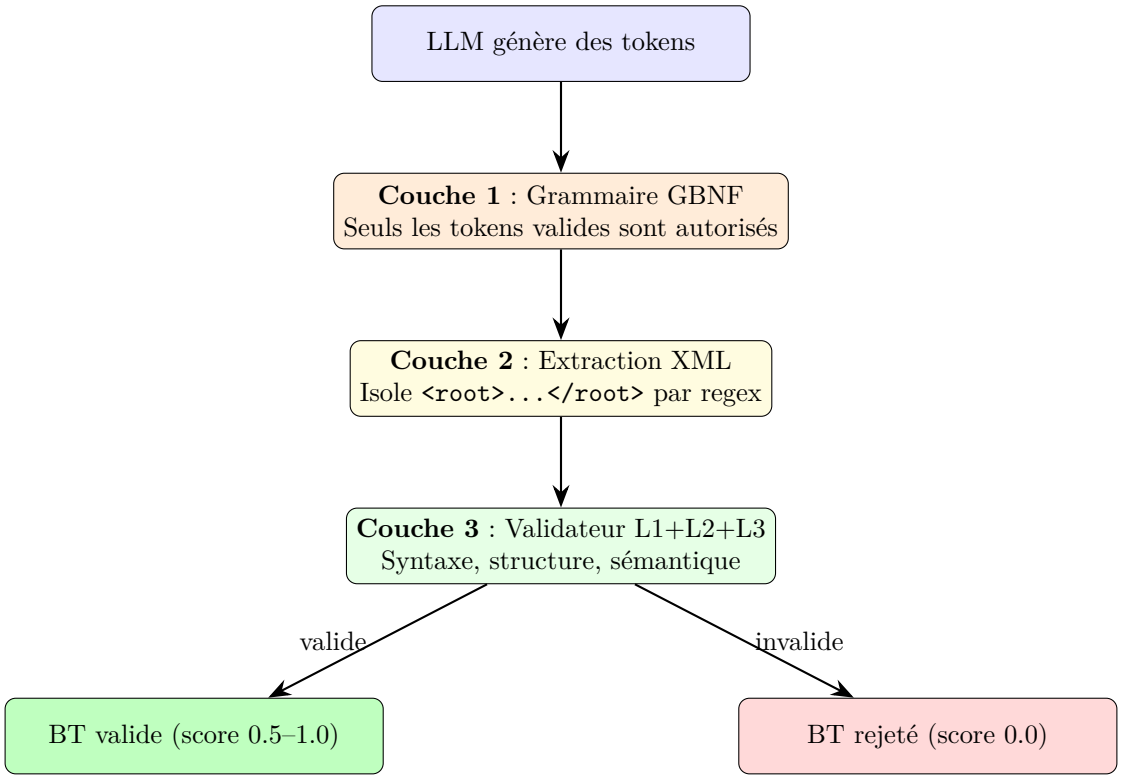

Figure 1 – Pipeline de défense en profondeur à trois couches

4.4 Couche 1 : Décodage contraint (niveau token)

Le décodage contraint empêche le LLM de générer des tokens qui violeraient la grammaire. À **chaque étape de génération** :

1. Le modèle propose une distribution de probabilités sur tous les tokens possibles.

2. La fonction `prefix_allowed_tokens_fn` vérifie quels tokens sont compatibles avec la grammaire GBNF.
3. Les tokens **incompatibles** reçoivent un score de $-\infty$ (ils ne seront jamais choisis).
4. Le modèle choisit parmi les tokens **restants** uniquement.

Implémentation : Le fichier `nav4rail_grammar.py` définit :

— `NAV4RAIL_GBNF` : la grammaire au format GBNF, listant explicitement les 27 noms de skills autorisés.

— `get_prefix_fn(tokenizer)` : crée la fonction de contrainte en utilisant la bibliothèque *lm-format-enforcer*[1], qui convertit un `RegexParser` en `prefix_allowed_tokens_fn`.

Garantie : *Zéro hallucination de nom de skill.* Il est **structurellement impossible** pour le modèle de générer un tag XML qui ne soit pas dans le catalogue des 27 skills.

Limite : La grammaire GBNF ne peut pas vérifier l'équilibrage des balises (propriété *context-sensitive*, hors de portée d'une grammaire régulière). Cette vérification est déléguée à la Couche 3.

4.5 Couche 2 : Extraction XML (post-traitement)

Après la génération, le texte brut peut contenir des artefacts (texte avant ou après le XML, répétitions). La fonction `_extract_xml()` (`finetune_lora_xml.py`, lignes 275–284) isole le bloc XML valide :

```
pattern = r"(<root\b.*?</root>)"
match = re.search(pattern, raw_output, re.DOTALL)
xml_str = match.group(1) if match else raw_output
```

Cette étape est simple mais essentielle : elle garantit que seul le contenu entre `<root>` et `</root>` est transmis au validateur.

4.6 Couche 3 : Validateur à trois niveaux

Le fichier `validate_bt.py` implémente un validateur en trois niveaux, détaillé dans la section 6.

Scoring :

— **1.0** : parfait (les 3 niveaux passés, aucun avertissement).

— **0.5–0.9** : valide avec avertissements (-0.1 par avertissement, plancher à 0.5).

— **0.0** : invalide (erreur bloquante au niveau L1 ou L2).

Références : `nav4rail_grammar.py`, `validate_bt.py`, `finetune_lora_xml.py` (lignes 275–284).

1. *lm-format-enforcer* : une bibliothèque Python qui transforme une expression régulière ou une grammaire GBNF en contrainte token par token compatible avec Hugging Face `model.generate()`.

5 Axe 4 — Fine-tuning LoRA : choix techniques

5.1 Contexte

L'objectif est d'entraîner le modèle **Mistral-7B-Instruct-v0.2** (7,3 milliards de paramètres) sur un cluster équipé de GPU **Tesla P100** (16 Go de VRAM) et **RTX 3090** (24 Go de VRAM).

Problème : un fine-tuning complet (tous les paramètres) de Mistral-7B nécessite environ 56 Go de VRAM — c'est-à-dire 3.5× la capacité d'un P100. La solution est **QLoRA**.

5.2 Décision : QLoRA 4-bit NF4

Au lieu de modifier les 7,3 milliards de paramètres, QLoRA :

1. **Quantifie** les poids du modèle de base en 4 bits NF4 (poids **gelés**, non modifiés pendant l'entraînement).
2. **Ajoute** de petites matrices entraînables (*adaptateurs LoRA*) à côté des poids gelés.

La formule de sortie effective est :

$$\mathbf{y} = \underbrace{W_0^{\text{4-bit, gelé}}}_{\text{modèle de base}} \cdot \mathbf{x} + \underbrace{\frac{\alpha}{r} \cdot B \cdot A}_{\text{adaptateur LoRA}} \cdot \mathbf{x} \tag{1}$$

où W_0 est la matrice de poids originale (gelée en 4 bits), A et B sont les petites matrices entraînables de rang r, et α est le facteur d'échelle.

5.3 Paramètres d'entraînement

5.4 Importance du masquage de la perte

Le `DataCollatorForCompletionOnlyLM` est un choix crucial : il masque la perte (*loss*) sur toute la partie *instruction* du prompt (avant le token `[/INST]`). Le modèle n'apprend **que** à générer le XML, pas à répéter le prompt système ou la description de mission.

Impact mesuré : convergence 2× plus rapide. Sans ce masquage, la perte d'évaluation (*eval_loss*) stagnait à ∼0.05 ; avec le masquage, elle descend à 0.017 dès la première version.

5.5 Choix du modèle

Mistral-7B-Instruct-v0.2 Modèle principal. Format d'instruction natif `[INST]...[/INST]`, bonnes performances en génération structurée. 7,3 milliards de paramètres.

TinyLlama-1.1B-Chat Modèle de référence (*baseline*). 1,1 milliard de paramètres, 6 minutes d'entraînement. Utile pour valider le pipeline avant de lancer un entraînement long sur Mistral.

Table 5 – Configuration QLoRA pour Mistral-7B

Paramètre	**Valeur**	**Explication**
Quantification	4-bit NF4 + double quant	Poids compressés en 4 bits, seconde quantification pour les constantes
Rang LoRA (r)	16	Dimension des matrices A et B — compromis entre expressivité et taille
Alpha LoRA (α)	32	Facteur d'échelle ($\alpha/r = 2$), amplifie l'influence des adaptateurs
Modules cibles	7 (q, k, v, o, gate, up, down)	Toutes les projections d'attention + couches MLP
Params. entraînables	42 M (0.58 % de 7.3 B)	Seule cette fraction est modifiée pendant l'entraînement
Optimiseur	`paged_adamw_8bit`	AdamW en 8 bits avec pagination mémoire (réduit la VRAM)
Planificateur LR	Cosine + 5 % warmup	Taux d'apprentissage décroît en cosinus après un échauffement
Taux d'apprentissage	2×10^{-4}	Valeur standard pour QLoRA
Masquage de la perte	`DataCollatorForCompletionOnlyLM`	La perte n'est calculée *que* sur la réponse XML (après le token `[/INST]`), pas sur le prompt
Taille de batch effective	64 (4 × 16)	4 exemples par GPU × 16 accumulations de gradient
Longueur max séquence	1 536 tokens	Couvre le prompt + BT généré (augmenté depuis 1 024 en v4)
VRAM utilisée	∼5.0 Go / 15.9 Go (P100)	∼31 % de la capacité

5.6 Résultats

Observation clé : le passage aux 27 skills réels (v4) a amélioré le score sémantique de 0.98 à **1.00** malgré la complexité accrue, car les patterns des skills réels sont plus réguliers que ceux des 8 skills proxy simplifiés.

Références : `finetune_lora_xml.py` (lignes 40–64 pour la configuration), `generate_dataset_v4.py`, `RESULTS.md`.

Table 6 – Évolution des performances par version du dataset

Configuration	Eval loss	Score L3	Warnings	Temps	GPU
TinyLlama, 100 ex.	0.118	n/a	—	6 min	P100
Mistral, 100 ex.	0.017	n/a	—	25 min	P100
Mistral, 500 ex. (v2)	0.010	0.97	3/10	140 min	P100
Mistral, 550 ex. (v3)	0.011	0.98	2/10	249 min	P100
Mistral, 550 ex. (v4)	**0.0035**	**1.00**	**0/10**	561 min	P100
Mistral, 2 000 ex. (v4 sc.)	0.0048	**1.00**	**0/15**	684 min	RTX 3090

6 Axe 5 — Évaluation : syntaxe, structure, sémantique

6.1 Contexte

Un simple *parse* XML (vérification syntaxique) est insuffisant : un XML syntaxiquement correct peut représenter un BT structurellement incohérent ou sémantiquement dangereux. Il faut pouvoir **discriminer la qualité** entre des BT valides (score 0.97 vs. 1.0) pour guider l'amélioration du dataset et du modèle.

6.2 Décision

Validateur à **trois niveaux** implémentés dans `validate_bt.py` :

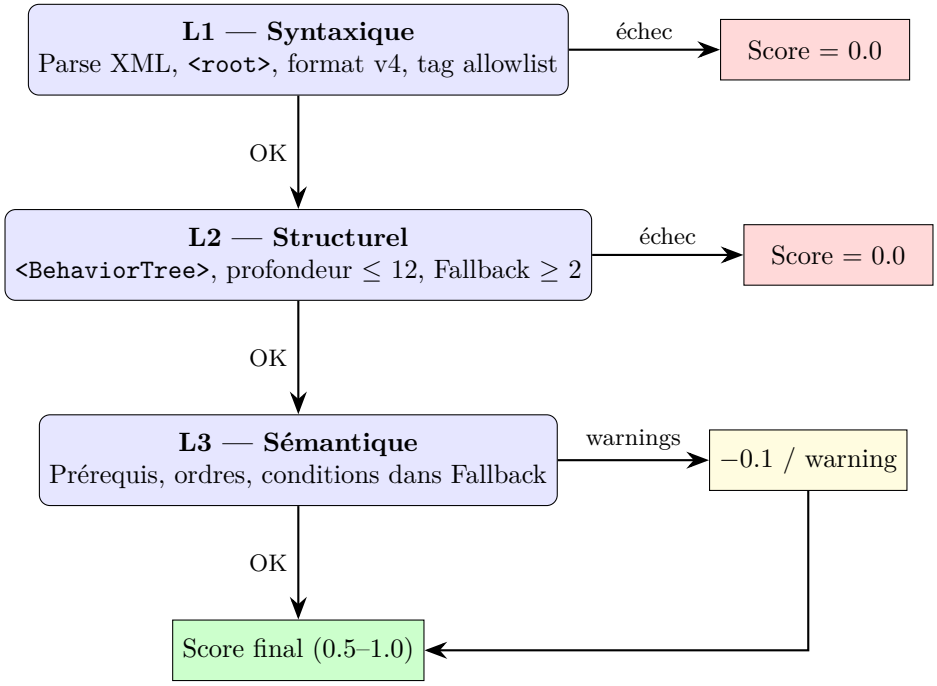

Figure 2 – Flux de validation à trois niveaux

6.3 Niveau L1 — Syntaxique (bloquant)

Ce niveau vérifie la **forme** du XML. Tout échec entraîne un score de **0.0** (BT rejeté).

— **Parse XML** : le texte doit être un XML bien formé (`xml.etree.ElementTree.fromstring`).

— **Tag racine** : doit être `<root>` avec l'attribut `BTCPP_format="4"`.
— **Liste blanche de tags** : chaque tag du BT doit appartenir à l'ensemble `VALID_TAGS` (32 tags : 27 skills + `root`, `BehaviorTree`, `Sequence`, `Fallback`, `Parallel`).
— **`<MoveAndStop/>` obligatoire** : tout BT doit contenir ce skill (le robot doit toujours s'arrêter à la fin de sa mission).

6.4 Niveau L2 — Structurel (bloquant)

Ce niveau vérifie la **structure** de l'arbre. Tout échec entraîne un score de **0.0**.

— `<BehaviorTree>` doit être présent sous `<root>`.
— Les nœuds de contrôle (`Sequence`, `Fallback`, `Parallel`) ne doivent **pas être vides**.
— Profondeur maximale : **12** niveaux d'imbrication (`MAX_DEPTH = 12`).
— Nombre maximal de skills : **35** nœuds (`MAX_SKILLS = 35`).
— Un `<Fallback>` doit avoir au moins **2 branches** (sinon il n'a aucun sens — un Fallback à 1 branche est juste une indirection inutile).

6.5 Niveau L3 — Sémantique (avertissements)

Ce niveau vérifie la **logique métier** du BT. Les erreurs à ce niveau ne rejettent pas le BT mais réduisent le score de -0.1 par avertissement (plancher à 0.5).

— **Prérequis** : certains skills doivent apparaître *avant* d'autres. Le dictionnaire `PREREQUISITES` définit 4 règles :

Skill	Doit être précédé de	Raison
`MissionStructureValid`	`LoadMission`	On ne peut valider une mission avant de la charger
`CreatePath`	`ProjectPointOnNetwork`	On ne peut créer un chemin sans projeter les points
`AgregatePath`	`CreatePath`	On ne peut agréger des chemins avant de les créer
`GenerateMissionSequence`	`LoadMission`	La séquence dépend de la mission chargée

— **Premier skill** : doit être `<LoadMission/>` (ou `<SimulationStarted/>` si la mission est une simulation).
— **Dernier skill** : doit être `<MoveAndStop/>` (tolérance : `<SignalAndWaitForOrder/>` peut le suivre).
— **Conditions dans Fallback** : les skills de type *condition* (`MissionTerminated`, `MissionFullyTreated`, `MeasurementsQualityValidated`, `MeasurementsEnforcedValidated`) doivent être à l'intérieur d'un nœud `<Fallback>`. En effet, ces conditions servent de « garde » pour quitter une boucle — elles n'ont de sens que dans un contexte de branchement.

6.6 Protocole de test

L'évaluation utilise **15 missions de test** non présentes dans le jeu d'entraînement, réparties en trois niveaux de difficulté :

— **5 missions classiques** : navigation simple, inspection standard.

— **5 missions complexes / hors distribution** : simulation avec inspection corrective, navigation multi-segments avec mesures.
— **5 missions ambiguës** : formulations volontairement imprécises pour tester la robustesse.

Meilleur résultat : sur le modèle v4 scaled (2 000 exemples), les **15/15 missions** obtiennent un score de **1.0** avec **0 avertissement**.

Baseline zero-shot : sans fine-tuning, Mistral-7B produit **0/10 BT valides** (score 0.0) — le modèle ne connaît pas le format BehaviorTree.CPP v4 et hallucine des tags génériques.

Références : `validate_bt.py`, `finetune_lora_xml.py` (lignes 339–422).

7 Tableau récapitulatif

Table 7 – Synthèse des cinq décisions d'architecture

Axe	**Décision**	**Alternative écartée**	**Justification clé**	**Fichier**
Proxy	Dataset synthétique (2 000 ex. à partir d'1 XML réel)	Annotation manuelle, données réelles	Coût prohibitif, BT compositionnels → génération par blocs	`generate_dataset_v4.py`
RAG	Rejeté (injection statique)	RAG vectoriel, injection par règles, few-shot	27 skills = 270 tok = 7 % contexte, seuil > 50 non atteint	`RAG_ARCHITECTURE.md`
Sécurité	Défense en profondeur 3 couches	Validation simple (parse XML seul)	GBNF = 0 hallucination + validateur 3 niveaux = BT sûrs	`nav4rail_grammar.py`
Fine-tuning	QLoRA 4-bit NF4, Mistral-7B, 0.58 % params	Fine-tuning complet (∼56 Go), GPT-2, Llama-3	P100 16 Go, format `[INST]` natif	`finetune_lora_xml.py`
Évaluation	Validateur L1+L2+L3, score 0.0–1.0	Parse XML simple	Distingue erreurs syntaxiques / structurelles / sémantiques	`validate_bt.py`

8 Catalogue des 27 skills NAV4RAIL

Pour référence, voici les 27 skills organisés par famille :

Table 8 – Catalogue des skills NAV4RAIL par famille

Famille	Skill	Type
Préparation (12)	LoadMission	Action
	MissionStructureValid	Condition
	UpdateCurrentGeneratedActivity	Action
	ProjectPointOnNetwork	Action
	CreatePath	Action
	AgregatePath	Action
	MissionFullyTreated	Condition
	PassAdvancedPath	Action
	PassMission	Action
	GenerateMissionSequence	Action
	GenerateCorrectiveSubSequence	Action
	InsertCorrectiveSubSequence	Action
Motion (9)	MissionTerminated	Condition
	CheckCurrentStepType	Condition
	PassMotionParameters	Action
	Move	Action
	UpdateCurrentExecutedStep	Action
	Deccelerate	Action
	MoveAndStop	Action
	SignalAndWaitForOrder	Action
	IsRobotPoseProjectionActive	Condition
Inspection (5)	ManageMeasurements	Action
	AnalyseMeasurements	Action
	MeasurementsQualityValidated	Condition
	PassDefectsLocalization	Action
	MeasurementsEnforcedValidated	Condition
Simulation (1)	SimulationStarted	Condition

Références des fichiers

`finetune_lora_xml.py` Pipeline d'entraînement QLoRA et d'inférence (496 lignes).

`validate_bt.py` Validateur de Behavior Trees à 3 niveaux (300 lignes).

`nav4rail_grammar.py` Grammaire GBNF et contrainte de décodage (198 lignes).

`generate_dataset_v4.py` Générateur du dataset synthétique v4 (2 000 exemples).

`RAG_ARCHITECTURE.md` Analyse détaillée du choix fine-tuning vs. RAG (266 lignes).

`SKILLS_CATALOG.md` Catalogue de référence des 27 skills NAV4RAIL.

`RESULTS.md` Résultats expérimentaux détaillés de toutes les versions.

`README.md` Documentation générale du pipeline.

`job_finetune.sh` / `job_finetune_v4.sh` Scripts SLURM d'orchestration sur le cluster.